Pertemuan 10
Klasifikasi dengan K-Nearest Neighbors

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Triyana Muliawati, S.Si., M.Si.
Rifky Fauzi

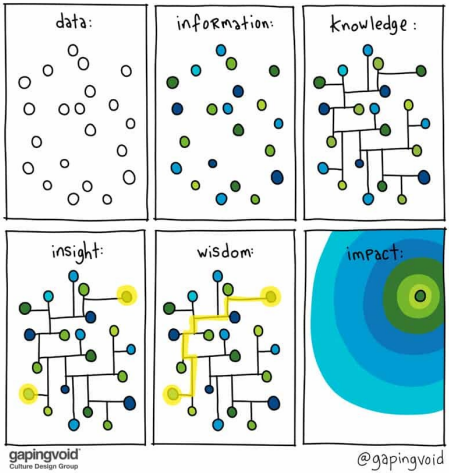
Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

Pokok bahasan

- Klasifikasi sebagai masalah prediksi kategori.
- Representasi fitur dan label kelas.
- Jarak Euclidean dan konsep tetangga terdekat.
- Algoritma K-Nearest Neighbors (KNN).
- Implementasi KNN dari prinsip matematika dengan NumPy.
- Kompleksitas algoritma, skala fitur, dan evaluasi.

Dari Data ke Keputusan



Ilustrasi: Gapingvoid Culture Design Group (gambar terlampir).

Dari titik menjadi kelas

Pada gambar, titik yang semula belum dibedakan mulai memperoleh **label** dan **hubungan**. Dalam klasifikasi, label dapat berupa warna, kategori, atau nama kelas.

Pertanyaan komputasi

Jika beberapa objek sudah diketahui kelasnya, dapatkah kita menentukan **kelas objek baru** berdasarkan objek yang sudah ada?

Hubungan dengan pertemuan sebelumnya

Regresi memprediksi **nilai numerik**; klasifikasi memprediksi **kelas/kategori**.

Klasifikasi: Apa yang Diprediksi?

Regresi pada pertemuan sebelumnya

Output model berupa nilai numerik:

$$\hat{y} \in \mathbb{R}.$$

Contoh:

- prediksi nilai ujian;
- estimasi harga rumah;
- perkiraan konsumsi listrik.

Klasifikasi

Output model berupa kategori:

$$\hat{y} \in \{C_1, C_2, \ldots, C_m\}.$$

Contoh:

- email: spam / bukan spam;
- transaksi: fraud / normal;
- objek: kelas A / B / C.

Gagasan utama

Klasifikasi menggunakan pola pada data berlabel untuk menentukan label data baru. Ini merupakan bentuk **supervised learning**.

Masalah Klasifikasi Dua Fitur

Contoh sederhana

Misalkan setiap mahasiswa direpresentasikan oleh

$$x = (x_1, x_2),$$

dengan x_1 = persentase kehadiran dan x_2 = nilai tugas. Labelnya

$$y \in \{\text{Lulus}, \text{Belum}\}.$$

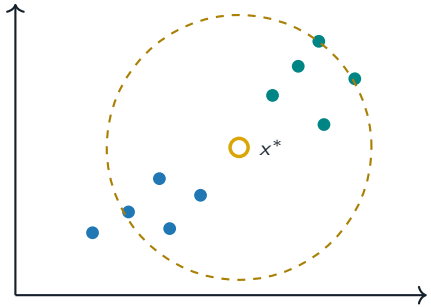
Data yang sudah ada mempunyai fitur dan label. Untuk mahasiswa baru, fitur diketahui tetapi label belum diketahui.



Coba tebak sebelum menghitung

Kelas mana yang Anda pilih untuk titik kuning? Alasan paling sederhana biasanya: **lihat objek-objek yang paling dekat**.

Intuisi Tetangga Terdekat



Cara manusia menebak

Untuk titik baru x^*, kita melihat titik-titik yang paling dekat. Jika sebagian besar tetangga terdekat berasal dari Kelas A, kita cenderung menempatkan x^* ke Kelas A.

Dari intuisi menjadi algoritma

Kita perlu menentukan:

1. apa arti "dekat";
2. berapa banyak tetangga yang dipakai;
3. bagaimana memilih kelas dari tetangga tersebut.

K-Nearest Neighbors untuk Klasifikasi

Diberikan data latih

$$D = \{(x_i, y_i)\}_{i=1}^{n}, \qquad x_i \in \mathbb{R}^d, \quad y_i \in \{C_1, \ldots, C_m\}.$$

Untuk objek baru x^*, KNN menentukan kelas menggunakan k objek data latih yang jaraknya paling dekat dengan x^*.

Masukan

- matriks fitur $X \in \mathbb{R}^{n \times d}$;
- label y sebanyak n buah;
- objek baru $x^* \in \mathbb{R}^d$;
- bilangan bulat k.

Keluaran

Kelas dengan jumlah suara terbesar di antara k tetangga terdekat:

$$\hat{y}(x^*) = \arg\max_{c} \sum_{i \in N_k(x^*)} \mathbf{1}(y_i = c).$$

Ciri penting

KNN tidak mencari parameter seperti β_0, β_1 pada regresi. Data latih disimpan dan digunakan ketika prediksi dilakukan.

Jarak Euclidean

Dari Teorema Pythagoras

Untuk dua titik dua dimensi

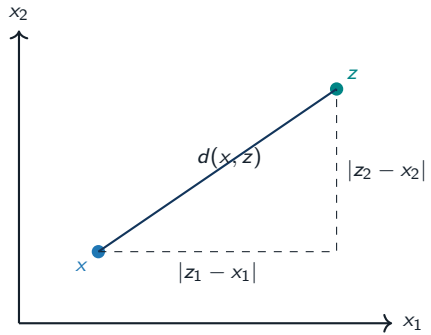
$$x = (x_1, x_2), \qquad z = (z_1, z_2),$$

jaraknya adalah

$$d(x, z) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$

Untuk d fitur:

$$d(x, z) = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2}$$



Makna

Semakin kecil jarak, semakin mirip posisi dua objek dalam ruang fitur.

Algoritma KNN untuk Klasifikasi

Input

Data latih (X, y), objek baru x^*, dan jumlah tetangga k.

1. Hitung jarak $d_i = d(x_i, x^*)$ untuk setiap data latih $i = 1, \ldots, n$.
2. Urutkan data berdasarkan d_i dari yang terkecil.
3. Ambil indeks k data pertama: $N_k(x^*)$.
4. Ambil label dari k tetangga tersebut.
5. Hitung banyak kemunculan setiap label.
6. Pilih label dengan frekuensi terbesar sebagai $\hat{y}(x^*)$.

Yang perlu dapat ditelusuri

Mahasiswa harus dapat menjelaskan: jarak mana yang dihitung, urutan tetangga, k indeks terpilih, jumlah suara tiap kelas, dan keputusan akhir.

Contoh Perhitungan Manual

Gunakan dua kelas, A dan B, dengan objek baru $x^* = (3.1, 2.7)$ dan $k = 3$.

Objek	Titik	Kelas	$d(x_i, x^*)$
1	$(1, 1)$	A	2.70
2	$(2, 1)$	A	2.03
3	$(2, 2)$	A	1.30
4	$(4, 4)$	B	1.58
5	$(5, 4)$	B	2.30
6	$(4, 5)$	B	2.47

Tiga jarak terkecil

Objek 3 (A), objek 4 (B), objek 2 (A).

Voting

A : 2 suara, B : 1 suara.

Maka

$$\boxed{\hat{y}(x^*) = \text{A}}$$

Perhatikan

KNN tidak "membuat garis" terlebih dahulu. Keputusan ditentukan dari posisi x^* terhadap data yang sudah berlabel.

Pengaruh Nilai k

$k = 1$

Keputusan hanya mengikuti satu tetangga terdekat.

- sangat lokal;
- sensitif terhadap noise atau data pencilan.

k sedang

Beberapa tetangga ikut memberikan suara.

- keputusan lebih stabil;
- pola lokal masih dipertahankan.

k terlalu besar

Banyak data yang jauh ikut memengaruhi keputusan.

- pola lokal dapat hilang;
- kelas mayoritas global dapat mendominasi.

Catatan praktis

Untuk dua kelas, k ganjil sering dipilih untuk mengurangi peluang hasil voting seri. Nilai k tetap perlu dievaluasi pada data yang tidak digunakan sebagai tetangga "jawaban" saat pemilihan model.

Implementasi KNN dari Prinsip Matematika

```python
import numpy as np

def knn_predict(X, y, x_new, k=3):
    # 1. jarak Euclidean ke semua data
    diff = X - x_new
    jarak = np.sqrt(np.sum(diff**2, axis=1))

    # 2. urutkan jarak, ambil k terdekat
    idx = np.argsort(jarak)[:k]
    label_tetangga = y[idx]

    # 3. voting mayoritas
    label, jumlah = np.unique(
        label_tetangga,
        return_counts=True
    )
    prediksi = label[np.argmax(jumlah)]

    return prediksi, jarak, idx
```

Yang terjadi di dalam kode

1. pengurangan vektor fitur;
2. kuadrat dan penjumlahan;
3. akar kuadrat;
4. sorting indeks;
5. penghitungan frekuensi label.

Bukan black box

Kita belum memakai pustaka pemodelan. NumPy hanya membantu menjalankan operasi matematis dan algoritmik yang sudah dipahami.

Tracing Implementasi KNN

```
X = np.array([
    [1, 1], [2, 1], [2, 2],
    [4, 4], [5, 4], [4, 5]
], dtype=float)

y = np.array(["A", "A", "A",
              "B", "B", "B"])

x_new = np.array([3.1, 2.7])

pred, jarak, idx = knn_predict(
    X, y, x_new, k=3
)

print(jarak)
print(idx)
print(y[idx])
print(pred)
```

Hasil yang harus dapat dijelaskan

- ‘jarak‘: satu nilai untuk setiap baris X;
- ‘idx‘: indeks tiga jarak terkecil;
- ‘y[idx]‘: label para tetangga;
- ‘pred‘: hasil voting mayoritas.

Pertanyaan code defense

Jika k diubah dari 3 menjadi 5, bagian mana dari algoritma yang berubah dan bagian mana yang tetap sama?

Kompleksitas Algoritma KNN

Misalkan terdapat n data latih dan d fitur. Untuk implementasi sederhana yang memakai `argsort`:

Langkah	Operasi	Big-O
Hitung jarak	$n \times d$ komponen	$O(nd)$
Urutkan jarak	sorting n nilai	$O(n \log n)$
Ambil tetangga	k indeks	$O(k)$
Voting	k label	$O(k)$

Total prediksi sederhana

$$T(n, d, k) = O(nd + n \log n + k).$$

Jika d dianggap tetap:

$$T(n) = O(n \log n).$$

Konsekuensi

KNN sederhana pada tahap "belajar", tetapi prediksi tetap perlu menghitung jarak terhadap data latih.

Skala Fitur Memengaruhi Jarak

Masalah

Misalkan dua fitur adalah:

- usia: sekitar 18–60;
- pendapatan: sekitar 2,000,000–20,000,000.

Pada jarak Euclidean, fitur dengan skala sangat besar dapat mendominasi hasil jarak.

Standardisasi

Salah satu pendekatan adalah mengubah fitur menjadi

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

dengan μ_j rata-rata dan s_j simpangan baku fitur ke-j pada data latih.

Prinsip penting

Parameter skala dihitung dari **data latih**, lalu digunakan kembali untuk mentransformasi data uji atau objek baru.

Evaluasi Hasil Klasifikasi

Data latih dan data uji

Data latih dipakai sebagai tetangga berlabel. Data uji digunakan untuk memeriksa apakah aturan KNN juga bekerja pada objek yang tidak dipakai sebagai acuan keputusan.

Akurasi

$$\text{akurasi} = \frac{\text{jumlah prediksi benar}}{\text{jumlah data uji}}.$$

Matriks kebingungan dua kelas

	Pred. A	Pred. B
Aktual A	benar A	salah ke B
Aktual B	salah ke A	benar B

Bukan hanya nilai akurasi

Periksa juga jenis kesalahan: kelas mana yang sering tertukar dan apakah jumlah anggota tiap kelas seimbang.

Latihan: KNN dari Perhitungan hingga Algoritma

Gunakan data berikut.

No	x_1	x_2	Kelas
1	1	1	A
2	2	1	A
3	2	2	A
4	3	2	A
5	3	3	A
6	5	4	B
7	5	5	B
8	6	4	B
9	6	5	B
10	7	5	B

Tugas

1. Untuk $x^* = (4.2, 3.2)$, hitung seluruh jarak Euclidean.
2. Urutkan jarak dan tentukan prediksi untuk $k = 3$.
3. Ulangi untuk $k = 5$. Apakah hasilnya berubah?
4. Implementasikan fungsi KNN menggunakan NumPy tanpa pustaka pemodelan.
5. Tampilkan indeks tetangga dan labelnya sebagai bagian tracing.
6. Tuliskan $T(n, d, k)$ dan Big-O algoritma yang digunakan.

Tujuan

Bukan sekadar memperoleh kelas akhir, tetapi dapat menjelaskan hubungan antara rumus jarak, sorting, voting, kode, dan kompleksitas algoritma.

Ringkasan

Klasifikasi

Memetakan objek dengan fitur x ke salah satu kelas yang tersedia.

Komponen matematis

Ruang fitur, jarak Euclidean, urutan jarak, dan voting mayoritas.

KNN

Keputusan didasarkan pada k data berlabel yang paling dekat menurut suatu ukuran jarak.

Komponen algoritmik

Perulangan/operasi vektor untuk jarak, sorting, pemilihan k indeks, counting label, serta analisis kompleksitas.

Inti pembelajaran

KNN menunjukkan bahwa sebuah metode machine learning dapat dipahami sebagai rangkaian operasi matematika dan algoritma yang sederhana, transparan, dan dapat ditelusuri.

Bacaan: James et al., *An Introduction to Statistical Learning*, 2nd ed.; Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, 2nd ed.